

Security Audit Report

Maintenance MES — FastAPI Backend (Phase2) + React Frontend

Toyota Boshoku Device India Pvt. Ltd. • Bawal, Haryana

Date: 2026-07-16

Scope: Phase2/ (main.py, auth.py, database.py, routers/) + mes-frontend/src

Type: Defensive
audit (own code)

Executive Summary

A full security review of the Maintenance MES was performed. The single most severe issue — the entire application source directory (including `.env` with live API keys and the SMTP password) being served unauthenticated over `/static` — combined with a hard-coded JWT signing key, gave a trivial unauthenticated path to full admin. **5 issues were fixed immediately in code** (2 files). The remaining items below require an operational action (e.g. rotating leaked credentials) or a product decision (the role/permission model) and are documented here to be scheduled and fixed.

5

FIXED IN CODE

4

CRITICAL total

3

HIGH pending

8

MED / LOW pending

A. Issues Fixed in Code (verified: backend compiles)

ID	Sev	Issue	Location	Fix applied
C1	CRIT	Whole source dir served at /static (.env, backups, logs, *.py downloadable, no auth)	main.py:79	Broad mount removed; only whitelisted image extensions served, path-traversal blocked
C3	CRIT	CORS allow_origins=* with allow_credentials=True (cross-site read of auth responses)	main.py:86	allow_credentials=False (auth is Bearer-header, not cookies)
C4	CRIT	AI chat SQL tool: startswith('SELECT') bypassable (multi-stmt writes, dump mes_admin)	main.py:1323	Single-SELECT only, write/DDL/file-func blocklist, sensitive tables blocked, READ ONLY txn
C2	CRIT	JWT SECRET_KEY hard-coded in source -> anyone can forge an admin token	auth.py:26	Now read from JWT_SECRET_KEY env; loud warning if fallback used (see OPS-2 to complete)
M2	MEDI	Mail header (CRLF) injection via user alert_level in notify-bypass Subject	main.py:2084	alert_level whitelisted to CRITICAL / WARNING

B. Pending Issues — to fix later

Grouped by priority. Each needs either an **operational action** (Ops) or a **product/role decision** before code changes.

■ Urgent — Operational (cannot be done from code)

OPS-1	CRITICAL	Rotate all leaked credentials — treat as compromised
Where	<code>.env</code> (ANTHROPIC / GOOGLE / OPENAI API keys, SMTP_PASS) + <code>database.py</code> DB password	
Risk	These lived in <code>.env</code> which was downloadable via <code>/static/.env</code> (C1) until the fix, so must be assumed leaked.	
Exploit	Attacker with the keys can run up LLM billing and send mail as the corporate mailbox <code>tbd_i_dx@toyota-boshoku.com</code> .	
Fix	Rotate all 3 LLM API keys, the Office365 SMTP password, and the Postgres password. Update <code>.env</code> with new values.	

OPS-2	CRITICAL	Set JWT_SECRET_KEY in .env to a random value
Where	<code>.env</code> (new var) — consumed by <code>auth.py:26</code>	
Risk	Until set, auth falls back to the built-in key that is visible in source, so admin tokens can still be forged.	
Exploit	<code>jwt.encode({'sub': '', 'role': 'admin', ...}, '')</code> -> full admin, no password.	
Fix	Generate: <code>python -c "import secrets; print(secrets.token_urlsafe(48))"</code> and set JWT_SECRET_KEY. Existing sessions re-login once.	

OPS-3	HIGH	Move .env, logs and DB/CSV backups out of the app folder
Where	Phase2/ <code>.env</code> , <code>*.log</code> , <code>pm_backup_*.csv</code> , <code>resolved_breakdowns_backup.csv</code> , etc.	
Risk	C1 now blocks serving them, but secrets/data dumps should not live inside a web-served directory at all.	
Fix	Relocate secrets to an env file outside the served tree; move backups to a non-served backups/ path; add them to <code>.gitignore</code> .	

OPS-4	MEDIUM	Remove hard-coded DB password fallback
Where	<code>database.py:27 os.getenv('DB_PASS', 'tbd_i@123')</code>	
Risk	A real DB password sits in source as a default; anyone with the file can read it.	
Fix	Drop the literal fallback (require DB_PASS from env). One-line change once <code>.env</code> reliably sets DB_PASS.	

■ High — needs a role / auth decision

H3	HIGH	Broken access control — mutations open to any logged-in role
Where	<code>pm.py:1239</code> (mail-config), quality approvals, breakdowns, daily_plan, etc.	
Risk	Many POST/PUT/DELETE use only <code>get_current_user</code> , so an 'operator' can hit config / approval / sign-off endpoints. Frontend <code>canAccess</code> is cosmetic; the backend is the real gate and does not enforce roles/permissions here.	
Exploit	Low-privilege user calls the API directly (curl/devtools) to change PM mail recipients, approve deviations, etc.	
Fix	Add a server-side <code>require_role('admin','plant_head','department')</code> dependency and/or enforce <code>mes_user_page_permissions</code> . NOTE: cannot blanket-apply <code>require_admin</code> because department users legitimately edit the Maintenance Panel — a role matrix decision is needed first.	

H1	HIGH	Unauthenticated poka-yoke (safety) endpoints
Where	poka_yoke.py:1729 events/ingest, :1699 auto-ack, :1459 sensor-sweep; main.py:2046 notify-bypass	
Risk	No auth dependency at all; comments say 'localhost only' but nothing enforces it.	
Exploit	Any LAN host can inject fake bypass events (spam QA) or auto-ack real ones (hide a safety bypass).	
Fix	Add auth, OR bind these to 127.0.0.1 / require a shared collector token and verify request.client.host. Decide how the collector authenticates.	

H2	HIGH	Poka-yoke admin CRUD in main.py skips the admin gate
Where	main.py:1770-1906 models/master/assignments/import (get_current_user, not require_admin)	
Risk	Duplicates of the router endpoints (which correctly use require_admin) drop the check, so a non-admin can rewrite/delete PY safety config.	
Exploit	operator/department login deletes or edits poka-yoke master definitions via the main.py routes.	
Fix	Change to require_admin OR remove the duplicates in favour of the router versions. Tie the decision to H3's role matrix (department users may need write here).	

■ Medium / Low — defense-in-depth

M1	MEDIUM	Second-order SQL injection via db_table_name
Where	main.py:1063, 1691, 2033; oee_alarm.py:125; lines.py (many) – f"... FROM {table}"	
Risk	Table name comes from mes_lines.db_table_name, written verbatim from request body with no identifier validation, then interpolated into SQL.	
Exploit	An admin (or forged-admin token) stores 'foo; DROP TABLE mes_admin; --' as db_table_name; it executes on the next read.	
Fix	Validate db_table_name on write against <code>^[a-z_][a-z0-9_]*\$</code> and/or use psycopg2.sql.Identifier for dynamic table names.	

M3	MEDIUM	Collector script path built from line_code
Where	provisioner.py:494-515 collector_{safe_name}.py, then subprocess.Popen	
Risk	safe_name only replaces '-'/space (not '.' or '/'), and PLC config is templated into a Python file that is then executed. Admin-gated, so defense-in-depth.	
Exploit	Crafted line_code could traverse the write path (..) or, if config is interpolated as code, run arbitrary Python.	
Fix	Validate line_code against <code>^[a-z0-9_]+\$</code> ; ensure _build_collector_script injects config as data (JSON/repr), never raw code.	

F1	MEDIUM	Frontend: JWT in sessionStorage + no CSP
Where	context/AuthContext.jsx (mes_token in sessionStorage)	
Risk	The bearer token is readable by any script running on the page. No dangerouslySetInnerHTML exists today, so it is latent, not active.	
Fix	Add a server-side Content-Security-Policy header (script-src 'self') to blunt any future injected-script token theft; ideally move the JWT to an HttpOnly cookie.	

L1	LOW	CMS default credentials in source
Where	main.py:939 CMS_USER='admin', CMS_PASS='admin123' fallbacks	
Risk	Weak default creds ship in source for the CMS camera portal proxy.	
Fix	Require CMS_USER / CMS_PASS from env with no fallback.	

L2	LOW	Verbose error disclosure
Where	main.py:1405 f"AI error: {str(e)}" and similar	
Risk	Raw exception text returned to the client leaks internal detail (paths, SQL, versions).	
Fix	Log the exception server-side; return a generic message to the client.	

L3	LOW	Long-lived JWT, no revocation
Where	auth.py:29 TOKEN_EXPIRE_HOURS=12; role read from token claim	
Risk	12h tokens with no jti/blacklist; role is trusted from the token rather than re-read from DB.	
Fix	After rotating the secret (OPS-2), shorten expiry and re-read role from mes_admin at request time (or add a revocation list).	

Checked and found OK

Passwords hashed with bcrypt correctly; login and most queries are parameterized (%s). No os.system / eval / exec / pickle in app code; provisioner subprocess is list-form (no shell=True). No dangerouslySetInnerHTML, no hard-coded secrets, and no .env in the frontend bundle. target=_blank links carry rel=noreferrer. config / users / zones / departments admin endpoints correctly use require_admin.

Report generated for internal remediation planning. Severity: Critical > High > Medium > Low. Line numbers reflect the code state on 2026-07-16 (post-fix for Section A items).